

基于SOC机器学习检测平台的时间序列建模---可疑登录检测

原创 郭豪、陈嘉豪 腾讯安全智能 2月24日

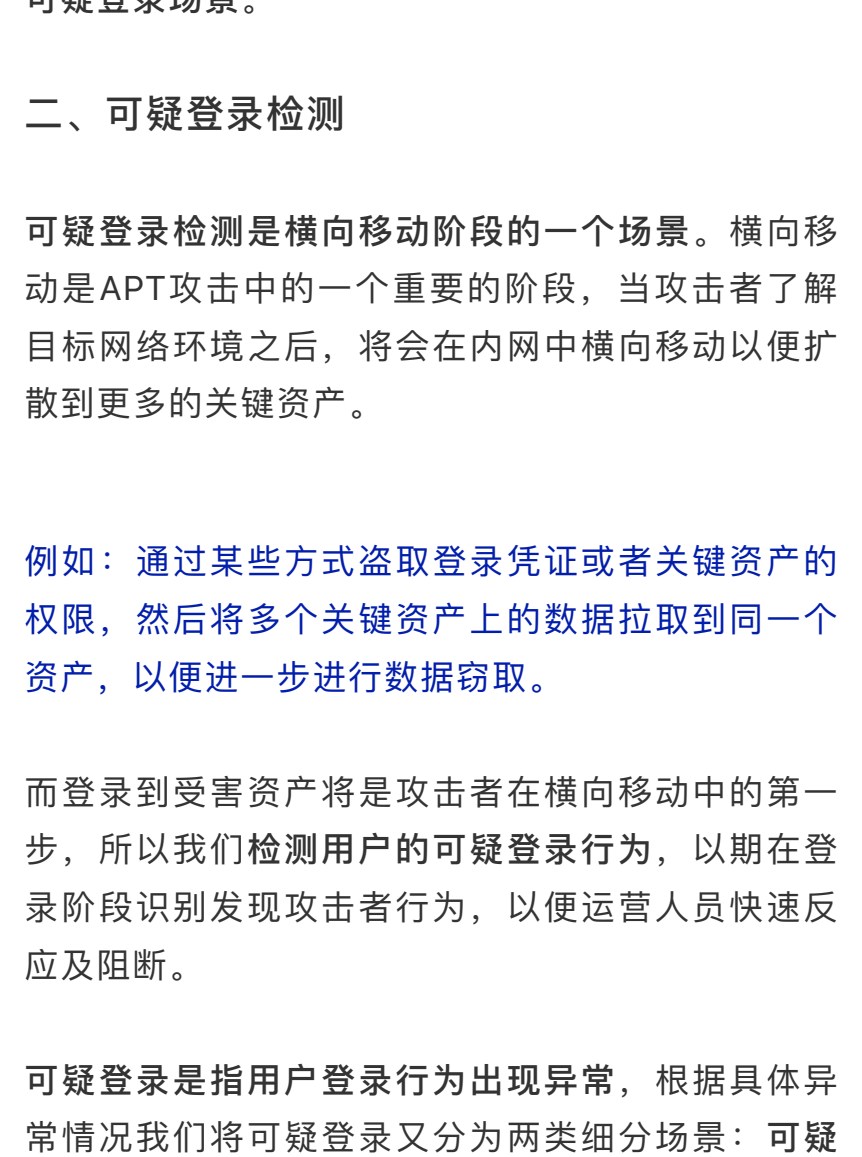
导语 基于Spark的SOC ML检测平台是企业安全运营人员打造的一站式机器学习检测服务平台，为用户提供从数据预处理、特征转换、特征提取、模型训练、模型预测、异常结果可视化到用户实时反馈的全流程可配置（无需编写代码）可直接运用于生产环境的智能威胁检测服务。本篇主要介绍基于SOC机器学习检测平台的时间序列建模的一个场景：可疑登录检测。

一、SOC机器学习检测框架及时间序列建模简述

企业里的威胁检测主要针对细分安全场景进行检测，场景多且不同企业关注场景可能不同。我们针对于企业威胁检测的特点构建了统一的SOC机器学习检测框架，框架的目的有两个：

- 是支撑SOC内置场景检测，丰富UEBA模块，增强SOC安全检测能力。
- 提供客户自定义场景功能，以满足客户自定义场景发现未知威胁的能力。

基于SOC机器学习检测框架的内置检测场景：



时间序列异常检测：

时间序列分析是将历史数据在时间维度上按先后顺序构建时间序列，并通过一系列相关模型进行建模识别潜在模式，以便对后续数据进行预测或者异常检测。

根据时间序列本身的特点，我们将时间序列分为周期性的时间序列和非周期性的时间序列。对不同类型的的时间序列应用不同的算法建模。例如：[周期性的时间序列应用STL](#)、[Holt-Winters等算法建模](#)；[非周期性的时间序列应用小波变换等算法建模](#)。

根据时间序列异常类型不同，我们将时间序列异常区分为离群点异常、拐点异常、子序列异常。

本文将介绍SOC机器学习框架支撑的一个内置场景检测：可疑登录检测。用户登录历史受白天晚上，周末工作日等影响一般有周期性规律，我们关注周期性所产生的影响，并期望将异常的点检出来，所以我们主要基于周期性建模+离群点异常检测识别可疑登录场景。

二、可疑登录检测

可疑登录检测是横向移动阶段的一个场景。横向移动是APT攻击中的一个重要的阶段，当攻击者了解目标网络环境之后，将会在内网中横向移动以便扩散到更多的关键资产。

例如：[通过某些方式盗取登录凭证或者关键资产的权限](#)，然后将多个关键资产上的数据拉取到同一个资产，以便进一步进行数据窃取。

而登录到受害资产将是攻击者在横向移动中的第一步，所以我们检测用户的可疑登录行为，以期在登录阶段识别发现攻击者行为，以便运营人员快速反应及阻断。

可疑登录是指用户登录行为出现异常，根据具体情况我们将可疑登录又分为两类细分场景：可疑失败登录、可疑首次登录（时间/位置）。

本文主要介绍从流量数据中识别检测可疑登录中的可疑失败登录。

三、检测方法

业界相关场景检测方法：

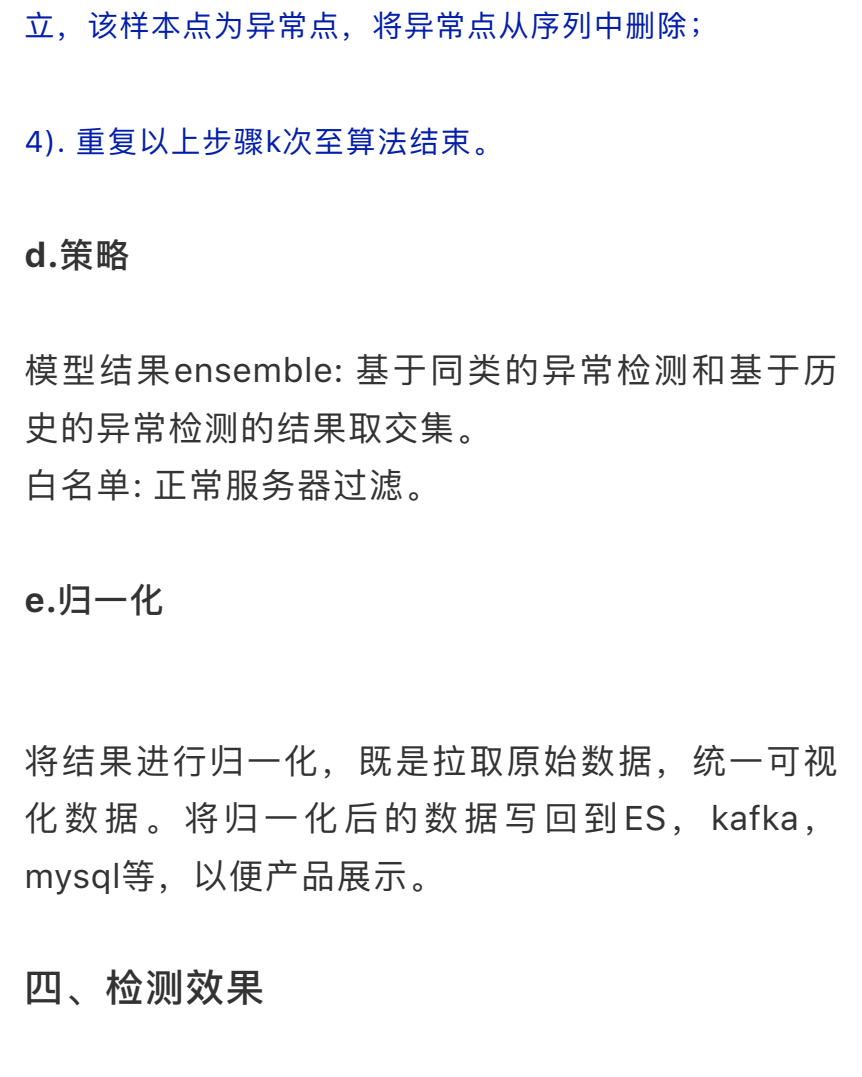
业界和可疑登录相关场景比如暴力破解的检测方法主要是基于规则：x时间段内（例如1min）登录失败次数达到N次被识别为暴力破解（例如：100次）。

基于上述规则检测可疑登录的优点：规则检测不需要训练、检测快。

基于上述规则检测可疑登录的缺点：

- 1.强依赖人工指定的阈值，容易漏检。阈值指定100次，99次就会被识别为正常登陆。
- 2.只通过单点检测，不结合用户历史行为，在客户环境下可能会因为业务配置等一些原因导致登录失败，并且不停尝试会造成大量的误报告警。

基于socML检测框架的方法：



基于SOC ML异常检测平台WEB画布配置如下：



我们基于soc ml检测框架检测可疑登录的流程如下：

a.流量数据预处理

- 过滤掉内对外流量，只保留内对内流量。既源IP为内网IP，目的IP为内网IP。
- 协议过滤：按不同协议分别检测，比如只过滤出mysql等 login流量日志。

b.特征提取

- 每个用户历史时间序列数据构建

X周登录数据：1min 为时间间隔，统计每个用户历史X周登录失败次数，并按时间进行排序，对缺失值进行填充处理。

- 同类用户数据构建

最新一段时间内同类数据：每个用户，在这y时间段内失败登录次数统计。

c.算法

我们从两个思路检测异常，两种思路的检测结果进行ensemble，得到最终的结果。

基于历史的行为异常检测：

我们打破规则单点检测的局限，对用户的过去一段时间的行为数据建模，当用户的行为偏离正常行为时，识别为异常。

基于同类的行为异常检测：

将用户在一段时间里的行为和其他同类行为比较，若用户此时的行为异于同类，识别为异常。

异常检测算法：时间序列异常检测算法使用STL + ESD, 行为分析异常检测算法使用HBOS。首先通过STL对时间序列进行建模分解，将时间序列分解为周期部分、趋势部分、残差部分，然后对残差数据用ESD进行检测异常。

STL：将时间序列分解为周期部分、趋势部分、残差部分。

$$R_X = X - S_X - T$$

四个变量分别代表时间序列、周期部分、趋势部分、残差部分。

Twitter的SHESD算法对STL做了优化将趋势部分用中值代替。本框架中算法采用twitter的实现方式。

具体示例如下图所示：



ESD：ESD是对Grubbs' Test的扩展，ESD可以检测上限k个异常值。假设定义如下：

H0: 数据集中没有异常值

H1: 数据集中有一个异常值

算法流程如下：

- 1). 计算与均值偏离最近的残差；

$$C_k = \frac{\max_k |x_k - \bar{x}|}{s}$$

- 2). 计算临界值（critical value）；

$$\lambda_k = \frac{(n-k)t_{\alpha, n-k-1}}{\sqrt{(n-k-1+t_{\alpha, n-k-1}^2)(n-k+1)}}$$

- 3). 检验原假设，比较检验统计量与临界值：若第1)步中计算得到的残差大于第2)步得到的临界值，则原假设H0不成立，该样本为异常点，将异常点从序列中删除；

- 4). 重复以上步骤k次至算法结束。

d.策略

模型结果ensemble: 基于同类的异常检测和基于历史的异常检测的结果取交集。
白名单: 正常服务器过滤。

e.归一化

将结果进行归一化，既是拉取原始数据，统一可视化数据。将归一化后的数据写回到ES, kafka, mysql等，以便产品展示。

四、检测效果

我们基于客户mysql/ssh等流量数据，检测可疑登录。结果显示：基于soc ml框架实现的可疑登录检测，能识别出数十次登录失败异常（算法根据历史建模自动识别异常，不受人工经验阈值影响）。同时对于业务配置引起的登录失败情况能较好的控制误报。

可疑登录示例：

Username: **_rental** Host: *.13*.10** Port: 3306
Time: 2019-07-07 12:31

同类对比：（异常）

历史对比：（异常）

